COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	10/14/26	**DATE SUBMITTED:**	10/15/26
NAME:	Pangyarihan,Jenny Rose Peñaflorida,Wrio Carlo	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

What procedures did you perform?
ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)
For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

ILOA

Strategy	Algorithm	Analysis
Recursion	Factorial or Fibonacci	The function calls itself on a smaller input. The base case (n <= 1) stops the calls. n - 1 works toward the base case, and return n * fact(n - 1) is the recursive call.
Brute Force	Linear search or bubble sort	Linear search checks every element one by one until it finds the target, without using any structure in the data. Bubble sort compares every adjacent pair, pass after pass, until nothing swaps.
Backtracking	N-Queens, maze solver, or subset-sum	It builds a solution step by step, such as placing one queen per row. If a placement violates a constraint, the algorithm undoes it (backs up) and tries the next option. The recursion is the state-space tree search.
Greedy	Dijkstra's, Prim's, or Kruskal's	At each step it picks the locally best candidate: the unvisited node with the smallest distance, or the cheapest edge that creates no

		cycle. It never revisits a choice.
Divide-and-Conquer	Merge sort, quick sort, or binary search	Merge sort divides the array into halves, solves each half recursively, and combines them with `merge()`. Binary search divides the range in half and discards one half each step.

Table 12-1. Algorithmic Strategies and Examples

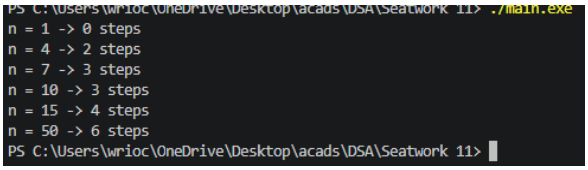
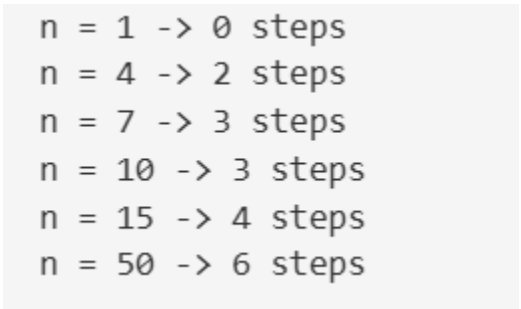
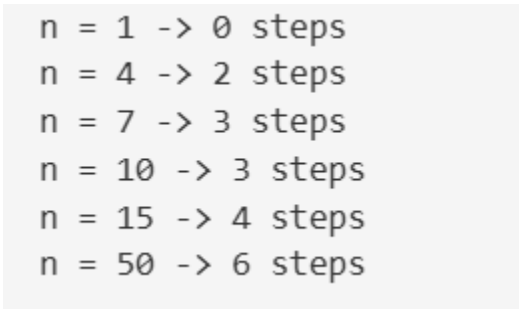
Screenshot	
Analysis	The outputs match the examples in the activity (1⟶0, 4⟶2, 7⟶3). For n = 10 it returns 3 (10⟶9⟶3⟶1), which beats the greedy answer of 4. Each value of n is computed once and then looked up, so the time is O(n) instead of the exponential time of plain recursion. The trade-off is O(n) extra memory plus recursion depth, which can overflow the stack for very large n.

Table 12-2. Memoization Implementation

Screenshot	
Analysis	The program ran without errors and showed the fewest steps needed to bring each test number down to 1: 0 for 1, 2 for 4, 3 for 7, 3 for 10, 4 for 15, and 6 for 50. It uses dynamic programming with an array where dp[i] stores the minimum steps for i. Since 1 needs no steps, dp[1] starts at 0. For every number after that, the code first tries subtracting 1, then

	checks whether dividing by 2 or by 3 (when possible) gives a smaller count, and keeps the best one. Because the answers for smaller numbers are already saved, nothing is calculated twice. For example, 10 takes 3 steps (10 → 9 → 3 → 1), which matches the output.

Screenshot	
Analysis	This also explains why greedy choices are not needed, since all three options are checked at every number. For example, 10 takes 3 steps (10 → 9 → 3 → 1), which matches the output.

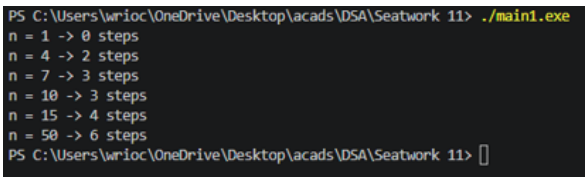
Screenshot	
Analysis	It starts from the trivial case `dp[1] = 0` and fills the table up to n, so every subproblem is already solved when it is needed. Time and space are both O(n). There is no recursion, so no stack overflow risk and less function-call overhead than memoization.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies
Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell
Description:

Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).
For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.
Deliverables:
Pseudocode (+ Solving Problem by Hand)

Cell	Possible path sums
(0,0)	4
(0,1), (0,2), (0,3)	11, 12, 18
(1,0)	10
(1,1)	17, 18
(1,2)	15, 20, 21
(1,3)	24, 27, 29, 30
(2,0)	13
(2,1)	21, 25, 26
(2,2)	16, 21, 22, 22, 26, 27
(2,3)	18, 23, 24, 24, 26, 28, 29, 29, 31, 32
(3,0)	20
(3,1)	21, 22, 26, 27
(3,2)	23, 28, 28, 29, 29, 29, 33, 33, 34, 34

Working C++ Code

Supplementary.cpp > solve(const vector<vector<int>>&, int)

```cpp
#include <iostream>
#include <vector>
using namespace std;

const int M = 4, N = 4;
vector<vector<vector<long long>>> memo;

long long countPaths(const vector<vector<int>>& mat, int i, int j, int cost) {
    if (i < 0 || j < 0 || cost < 0) return 0;
    if (i == 0 && j == 0) return (mat[0][0] == cost) ? 1 : 0;
    if (memo[i][j][cost] != -1) return memo[i][j][cost];

    int remaining = cost - mat[i][j];
    return memo[i][j][cost] =
        countPaths(mat, i - 1, j, remaining) +
        countPaths(mat, i, j - 1, remaining);
}

long long solve(const vector<vector<int>>& mat, int cost) {
    memo.assign(M, vector<vector<long long>>(N, vector<long long>(cost + 1, -1)));
    return countPaths(mat, M - 1, N - 1, cost);
}

int main() {
    vector<vector<int>> printed = {
        {4, 7, 1, 6}, {6, 7, 3, 9}, {3, 8, 1, 2}, {7, 1, 7, 3}};
    vector<vector<int>> classic = {
        {4, 7, 1, 6}, {5, 7, 3, 9}, {3, 2, 1, 2}, {7, 1, 6, 3}};

    cout << "Printed matrix, cost 25: " << solve(printed, 25) << "\n";
    cout << "Printed matrix, cost 26: " << solve(printed, 26) << "\n";
    cout << "Classic matrix, cost 25: " << solve(classic, 25) << "\n";
}
```

utput Debug Console **Terminal** Ports Spell Checker

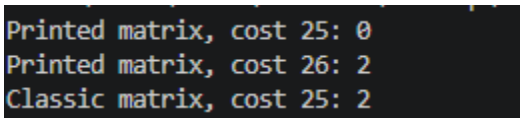
Terminal

```
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\Seatwork 11> g++ -g Supplementary.cpp -o Supplementary.exe
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\Seatwork 11> ./Supplementary.exe
Printed matrix, cost 25: 0
Printed matrix, cost 26: 2
Classic matrix, cost 25: 2
PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\Seatwork 11>
```

Analysis of working code

- Strategy: recursion plus memoization (top-down dynamic programming). The problem has overlapping subproblems, since the same (i, j, remaining cost) state is reached by many paths. It also has optimal substructure, because the count for a cell is the sum of the counts from the two cells that can lead into it.
- Base cases: the start cell returns 1 only if its value equals the remaining cost. Out-of-bounds or negative remaining cost returns 0.
- Complexity: without memoization it is O(2^(M+N)) in the worst case. With memoization it is O(M × N × cost) time and space, since each state is computed once.
- Limitation: memory grows with the target cost, so very large costs call for a hash map or a bottom-up table.

Screenshots of demonstration




```cpp
#include <iostream>
#include <vector>
using namespace std;

const int M = 4, N = 4;
vector<vector<vector<long long>>> memo;

long long countPaths(const vector<vector<int>>& mat, int i, int j, int cost) {
    if (i < 0 || j < 0 || cost < 0) return 0;
    if (i == 0 && j == 0) return (mat[0][0] == cost) ? 1 : 0;
    if (memo[i][j][cost] != -1) return memo[i][j][cost];

    int remaining = cost - mat[i][j];
    return memo[i][j][cost] =
        countPaths(mat, i - 1, j, remaining) +
        countPaths(mat, i, j - 1, remaining);
}

long long solve(const vector<vector<int>>& mat, int cost) {
    memo.assign(M, vector<vector<long long>>(N, vector<long long>(cost + 1, -1)));
    return countPaths(mat, M - 1, N - 1, cost);
}
```

```
roblems  Output  Debug Console  Terminal  Ports   Filter (e.g. text, !excludeText, t...

[Running] cd "c:\Users\jenny rose\Documents\pangyarihan dsa\HOA12.1\" && g++ tempCodeRunnerFile.cpp -o tem
rose\Documents\pangyarihan dsa\HOA12.1\"tempCodeRunnerFile
Printed matrix, cost 25: 0
Printed matrix, cost 26: 2
Classic matrix, cost 25: 2
```

Analysis of working code:

The program ran without errors and gave these results: 0 paths for the printed matrix at cost 25, 2 paths for the printed matrix at cost 26, and 2 paths for the classic matrix at cost 25. This shows the code only counts paths whose cell values add up to the exact cost, so it returns 0 when no path matches.

It works through recursion with memoization. Starting from the bottom-right cell, the function moves backward by going up or left, subtracting each cell's value from the cost along the way. When it reaches the top-left cell, it returns 1 if that cell's value equals what is left of the cost, and 0 if not. A path that goes outside the matrix or makes the cost negative is dropped right away. The memo table remembers answers already found, so the program never repeats the same calculation, and it is reset before each test so the results do not mix.

Conclusion:

Recursion is the base of most strategies. Brute force tries everything, backtracking prunes failed branches, and greedy takes the locally best choice. Divide-and-conquer splits a problem and combines the results, and dynamic programming saves repeated work. The minimum-steps problem showed that greedy can fail (n = 10), while DP guarantees the optimum. Memoization is more intuitive because it follows the recurrence directly. Bottom-up is faster in practice and avoids stack overflow. Both give identical results with O(n) time. Defining the state as (row, column, remaining cost) turned an exponential path search into a polynomial one. I also learned to verify the given examples against the data, since the sample answer didn't match the printed matrix.

Through this activity, I found out that memoization stores the results of smaller subproblems so the program never has to solve the same one twice, and that any recursive solution must have a well-defined base case. The procedure went as planned, and trying different inputs helped verify that the results were right. For the supplementary activity, the program counted how many paths from the top-left to the bottom-right of a 4x4 matrix have cell values that add up to an exact cost, relying on a 3D memo table (row, column, remaining cost) to avoid repeating calculations. I feel I did well since I understood the logic and got the program running, though I still need to get better at tracing recursion by hand and spotting bugs quickly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.